easier for the readers to compare them.

The initialization of the cache is almost identical to the LRU one, we just create an empty priority queue instead of a linked list.

When it comes to adding an entry, things get a bit trickier. Remember that we mentioned how you need to be careful in order to implement the priority right? We want to make sure that, among the elements with the minimum number of entries, the oldest is removed first, otherwise we could end up removing the latest entry over and over again, without giving it a chance to have its counter increased.

Listing 7.6 LFU Cache set

```
function LFUCache::set(key, value)
  if hashTable.contains(key) then

    node ← this.hashTable.get(key)
         #1

node.setValue(value)

    node.updatePriority(new Tuple(node.getCounter() + 1,
time()))    #2
    return
false
  elsif getSize() >= maxSize then
    evictOneEntry()
  newNode ← elements.add(key, value, new Tuple(1,
time()))           #3
         hashTable.set(key,
newNode)
  return true
```